

Orientação a Objetos PHP

Prof. Juliana de Sá

Baseado no material desenvolvido pelo Prof. Fabricio
Azevedo



A classe

- Devemos lembrar que uma classe é composta por atributos e métodos, ou seja, vamos definir suas características e suas ações.
- Primeiro passo é definir como será a classe e qual a sua funcionalidade, com isso, poderemos definir suas características e suas ações.

Operacao

- valor1 : double
- valor2 : double

- + somar() : double
- + subtrair() : double
- + dividir() : double
- + multiplicar() : double
- + exponenciar() : double

Classe Operacao

Criando uma Classe

- O nome do arquivo e o nome da classe devem ser o mesmo.
- Lembrando que toda classe deve começar com letra maiuscula

```
Operacao.php
1  <?php
2
3  class Operacao{
4
5
6  }
7
8  ?>
```

Criando os atributos

- Podemos criar nossos atributos de três formas diferentes, apenas utilizando as instruções **public**, **private** e **protected**,

1ª Forma – Declarações pública dos atributos

- Este atributo pode ser acessível por todos. Um método ou propriedade **public** é acessível através de um objeto instanciado, por outros métodos e por outras subclasses

 Operacao.php

```
1  <?php
2
3  class Operacao{
4      public $valor1;
5      public $valor2;
6  }
7
8  ?>
```

Exemplo de acesso

Testa.php

```
1  <?php
2  require_once ('Operacao.php');
3
4  $op1 = new Operacao();
5
6  $op1->valor1 = 2;
7
8
```

2ª Forma – Declarações privadas dos atributos

- Este atributo pode ser acessível somente para a própria classe. Um método ou propriedade **private** (privada) só é acessível de dentro da própria classe. Mesmo as subclasses não conseguirão acesso

```
Operacao.php
1  <?php
2
3  class Operacao{
4      private $valor1;
5      private $valor2;
6  }
7
8  ?>
```


Acesso aos atributos privados

- Ao acessarmos como fizemos no exemplo anterior teremos o seguinte erro:

```
Fatal error: Uncaught Error: Cannot access private property Operacao::$valor1  
in C:\xampp\htdocs\turmaA\Testa.php:6 Stack trace: #0 {main} thrown in  
C:\xampp\htdocs\turmaA\Testa.php on line 6
```

Acesso aos atributos privados

- O acesso a atributos privados deve ser feito através dos métodos set e get.

```
//armazena informação no atributo
public function setValor1($valor1){
    $this->valor1 = $valor1;
}

public function setValor2($valor2){
    $this->valor2 = $valor2;
}
```

```
//pega a informação armazenada no atributo
✓ public function getValor1(){
    $this->valor1 = $valor1;
}

✓ public function getValor2(){
    $this->valor2 = $valor2;
}
```

3ª Forma – Declarações protegidas dos atributos

- Este atributo pode ser acessível somente para a própria classe ou por subclasses. Um método ou propriedade definida como **protected** não é acessível de fora dela. Por exemplo, se você instanciar um objeto, não conseguirá invocar um método protected

🐘 Operacao.php

```
1  <?php
2
3  class Operacao{
4      protected $valor1;
5      protected $valor2;
6  }
7
```

Acesso aos atributos protegidos

- As classes e as subclasses podem acessar direto, já as outras classes o acesso deve ser feito através dos métodos set e get.

Criando Métodos na classe

- Para a criação de métodos (ações) devemos utilizar a instrução **function**. Na verdade, uma função é uma subrotina com uma determinada funcionalidade, e quando essa função tem algum retorno de informação utilizamos a **return**

Criando Métodos na classe

```
public function somar(){  
    return $this->valor1 + $this->valor2;  
}
```

Utilizando a Classe

- Primeiramente devemos criar uma página em PHP, onde será utilizada a classe **Operacao**.
- Nesta página devemos incluir o arquivo com a classe externa, ou seja, vamos utilizar a instrução **require_once()**

```
Testa.php
1  <?php
2  require_once ('Operacao.php');
3
4  $op1 = new Operacao();
5
6
7  ?>
```

Sempre que abrir uma nova página (.PHP) deverá ser incluída novamente a biblioteca (classe).

Instanciando a classe

(criando um objeto)

```
Testa.php  
1 <?php  
2 require_once ('Operacao.php');  
3  
4 $op1 = new Operacao();
```

`$op1 = new Operacao();`

Nome do
Objeto

Comando
para
instanciar

Construtor a
ser utilizado

Utilizando o objeto

- Após o objeto instanciado, agora devemos enviar os dados do usuário para o objeto (classe)

```
$op1->setValor1(10);  
$op1->setValor2(2.6);
```

Utilizando o objeto

- Após o envio de todos os dados necessário ao objeto, podemos solicitar os dados de retorno através dos métodos, nesse caso iremos utilizar o método **somar()**,

```
$resultado = $op1->somar();
```